

NumPy Quick Reference Guide

1. TẠO MẢNG

Từ dữ liệu có sẵn

```
python  
  
np.array([1, 2, 3])           # Từ list  
np.array([[1,2], [3,4]])     # Mảng 2D  
np.asarray(data)             # Chuyển đổi sang array
```

Tạo mảng đặc biệt

```
python  
  
np.zeros(5)                   # [0, 0, 0, 0, 0]  
np.zeros((3, 4))             # Ma trận 3x4 toàn 0  
np.ones((2, 3))              # Ma trận 2x3 toàn 1  
np.full((2, 2), 7)           # Ma trận 2x2 toàn số 7  
np.eye(3)                    # Ma trận đơn vị 3x3  
np.empty((2, 3))             # Mảng chưa khởi tạo
```

Dãy số

```
python  
  
np.arange(0, 10, 2)          # [0, 2, 4, 6, 8]  
np.linspace(0, 1, 5)         # 5 số từ 0 đến 1  
np.logspace(0, 2, 5)         # 5 số log từ 10^0 đến 10^2
```

Ngẫu nhiên

```
python  
  
np.random.random((3, 3))     # Float 0-1  
np.random.randint(0, 10, size=5) # Int từ 0-9  
np.random.randn(3, 3)        # Phân phối chuẩn  
np.random.choice([1,2,3], size=5) # Chọn ngẫu nhiên  
np.random.shuffle(arr)       # Xáo trộn
```

2. THUỘC TÍNH MẢNG

```
python

arr.shape      # Kích thước: (3, 4)
arr.ndim       # Số chiều: 2
arr.size       # Tổng phần tử: 12
arr.dtype      # Kiểu dữ liệu: int64
arr.itemsize   # Kích thước 1 phần tử: 8 bytes
arr.nbytes     # Tổng bytes: 96
```

3. TRUY CẬP & SLICING

```
python

arr[0]         # Phần tử đầu
arr[-1]        # Phần tử cuối
arr[1:4]       # Từ vị trí 1 đến 3
arr[:,2]       # Mọi phần tử thứ 2
arr[::-1]      # Đảo ngược

# 2D
arr[0, 1]      # Hàng 0, cột 1
arr[1, :]     # Toàn bộ hàng 1
arr[:, 2]      # Toàn bộ cột 2
arr[0:2, 1:3]  # Slice 2D
```

4. THAY ĐỔI HÌNH DẠNG

```
python

arr.reshape(3, 4)      # Đổi thành 3x4
arr.flatten()          # Về 1D (copy)
arr.ravel()            # Về 1D (view)
arr.T                  # Chuyển vị
np.transpose(arr)      # Chuyển vị
arr.resize((2, 6))     # Thay đổi tại chỗ
np.expand_dims(arr, axis=0) # Thêm chiều
np.squeeze(arr)        # Xóa chiều size=1
```

5. GHÉP & TÁCH MẢNG

```
python

np.concatenate([a, b])      # Nối theo axis=0
np.vstack([a, b])           # Ghép dọc
np.hstack([a, b])           # Ghép ngang
np.column_stack([a, b])     # Ghép thành cột
np.dstack([a, b])           # Ghép theo chiều depth

np.split(arr, 3)             # Chia thành 3 phần
np.hsplit(arr, 2)            # Chia ngang
np.vsplit(arr, 2)            # Chia dọc
```

6. TOÁN HỌC CƠ BẢN

```
python

# Toán tử
arr + 5          # Cộng scalar
arr1 + arr2      # Cộng từng phần tử
arr * 2          # Nhân scalar
arr1 * arr2      # Nhân từng phần tử
arr ** 2         # Lũy thừa

# Hàm
np.add(a, b)     # Cộng
np.subtract(a, b) # Trừ
np.multiply(a, b) # Nhân từng phần tử
np.divide(a, b)  # Chia
np.power(a, 2)   # Lũy thừa
np.sqrt(arr)     # Căn bậc 2
np.abs(arr)      # Giá trị tuyệt đối
np.exp(arr)      #  $e^x$ 
np.log(arr)      #  $\ln(x)$ 
np.log10(arr)    #  $\log_{10}(x)$ 
```

7. THỐNG KÊ

```
python
```

<code>np.sum(arr)</code>	<i># Tổng</i>
<code>np.sum(arr, axis=0)</code>	<i># Tổng theo cột</i>
<code>np.sum(arr, axis=1)</code>	<i># Tổng theo hàng</i>
<code>np.mean(arr)</code>	<i># Trung bình</i>
<code>np.median(arr)</code>	<i># Trung vị</i>
<code>np.std(arr)</code>	<i># Độ lệch chuẩn</i>
<code>np.var(arr)</code>	<i># Phương sai</i>
<code>np.min(arr)</code>	<i># Giá trị nhỏ nhất</i>
<code>np.max(arr)</code>	<i># Giá trị lớn nhất</i>
<code>np.argmin(arr)</code>	<i># Vị trí min</i>
<code>np.argmax(arr)</code>	<i># Vị trí max</i>
<code>np.percentile(arr, 50)</code>	<i># Phân vị 50%</i>
<code>np.quantile(arr, 0.75)</code>	<i># Quantile 75%</i>
<code>np.cumsum(arr)</code>	<i># Tổng tích lũy</i>
<code>np.cumprod(arr)</code>	<i># Tích tích lũy</i>

8. LOGIC & ĐIỀU KIỆN

python

So sánh

`arr > 5` *# Boolean array*

`arr == 3` *# Bằng*

`arr != 0` *# Khác*

Điều kiện

`np.where(arr > 5, 'Yes', 'No')` *# If-else*

`np.select([cond1, cond2], [val1, val2])`

Logic

`np.any(arr > 5)` *# Có ít nhất 1 True*

`np.all(arr > 0)` *# Tất cả đều True*

`np.logical_and(a, b)` *# AND*

`np.logical_or(a, b)` *# OR*

`np.logical_not(a)` *# NOT*

Kiểm tra

`np.isnan(arr)` *# Kiểm tra NaN*

`np.isinf(arr)` *# Kiểm tra Infinity*

`np.isfinite(arr)` *# Kiểm tra finite*

9. SẮP XẾP & TÌM KIẾM

python

`np.sort(arr)` *# Sắp xếp tăng dần*

`np.sort(arr[::-1])` *# Sắp xếp giảm dần*

`np.argsort(arr)` *# Vị trí sau khi sort*

`arr.sort()` *# Sort tại chỗ*

`np.unique(arr)` *# Giá trị duy nhất*

`np.unique(arr, return_counts=True)` *# + số lần xuất hiện*

`np.searchsorted(arr, 5)` *# Tìm vị trí chèn*

`np.nonzero(arr)` *# Vị trí khác 0*

10. ĐẠI SỐ TUYẾN TÍNH

python

```

# Nhân ma trận
np.dot(A, B)          # Tích vô hướng / Nhân ma trận
A @ B                # Nhân ma trận (Python 3.5+)
np.matmul(A, B)       # Nhân ma trận

# Ma trận
np.linalg.inv(A)       # Ma trận nghịch đảo
np.linalg.det(A)       # Định thức
np.linalg.eig(A)       # Trị riêng & vector riêng
np.linalg.matrix_rank(A) # Hạng ma trận

# Giải phương trình
np.linalg.solve(A, b)   # Giải  $Ax = b$ 
np.linalg.lstsq(A, b)   # Bình phương tối thiểu

# Chuẩn
np.linalg.norm(v)       # Chuẩn L2 của vector
np.linalg.norm(A, 'fro') # Chuẩn Frobenius

```

11. BROADCASTING

```

python

# NumPy tự động mở rộng mảng khi tính toán
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
arr + np.array([10, 20, 30]) # Cộng cho mỗi hàng

# Shape tương thích:
# (3, 4, 5) + (5,)    ✓ OK
# (3, 4, 5) + (4, 5)   ✓ OK
# (3, 4, 5) + (3, 1, 5) ✓ OK
# (3, 4, 5) + (4,)     ✗ Lỗi

```

12. COPY VS VIEW

```
python
```

```
# View (thay đổi ảnh hưởng gốc)
```

```
b = a[:]          # View
```

```
b = a.view()      # View
```

```
b = a.ravel()     # View
```

```
# Copy (độc lập với gốc)
```

```
b = a.copy()      # Deep copy
```

```
b = np.array(a)   # Copy
```

13. XỬ LÝ MISSING DATA

```
python
```

```
# Tạo NaN
```

```
arr = np.array([1, 2, np.nan, 4])
```

```
# Kiểm tra
```

```
np.isnan(arr)      # [False, False, True, False]
```

```
np.isnan(arr).sum() # Đếm số NaN
```

```
# Loại bỏ
```

```
arr[~np.isnan(arr)] # Lọc NaN
```

```
# Thay thế
```

```
np.nan_to_num(arr, nan=0) # NaN → 0
```

```
np.nanmean(arr)      # Mean bỏ qua NaN
```

```
np.nansum(arr)       # Sum bỏ qua NaN
```

14. KIỂU DỮ LIỆU

```
python
```

```
arr.dtype           # Xem kiểu hiện tại
arr.astype('float64') # Đổi kiểu
arr.astype(int)      # Đổi sang int
```

```
# Các kiểu phổ biến:
# int8, int16, int32, int64
# uint8, uint16, uint32, uint64
# float16, float32, float64
# bool, object, string_
```

15. FILE I/O

```
python

# Lưu & đọc file
np.save('array.npy', arr) # Lưu binary
arr = np.load('array.npy') # Đọc binary

np.savez('arrays.npz', a=arr1, b=arr2) # Nhiều array
data = np.load('arrays.npz')
arr1 = data['a']

# Text file
np.savetxt('data.csv', arr, delimiter=',')
arr = np.loadtxt('data.csv', delimiter=',')
```

16. TIPS & TRICKS

```
python
```



```
# Set options
np.set_printoptions(precision=3, suppress=True)

# Lặp qua từng phần tử
for x in np.nditer(arr):
    print(x)

# Vectorize function
def my_func(x):
    return x ** 2
vec_func = np.vectorize(my_func)
vec_func(arr)

# Memory usage
arr.nbytes / 1024 / 1024    # MB

# Copy từ list nhanh
arr = np.fromiter(my_list, dtype=int)
```

17. CÔNG THỨC THƯỜNG DÙNG

```
python

# Chuẩn hóa (0-1)
(arr - arr.min()) / (arr.max() - arr.min())

# Chuẩn hóa (mean=0, std=1)
(arr - arr.mean()) / arr.std()

# Khoảng cách Euclidean
np.sqrt(np.sum((a - b) ** 2))

# Cosine similarity
np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

TÀI LIỆU THAM KHẢO

- Official Docs: <https://numpy.org/doc/stable/>
- API Reference: <https://numpy.org/doc/stable/reference/>

- Cheat Sheet: https://numpy.org/doc/stable/user/absolute_beginners.html

Sử dụng `help(np.function_name)` để xem chi tiết từng hàm!